

Plataforma completa para aprender inglés con niveles CEFR (A1-C2)

[Demo en vivo](#) · [Reportar Bug](#) · [Solicitar Feature](#)

Tabla de Contenidos

- [Características](#)
- [Tecnologías](#)
- [Instalación](#)
- [Configuración](#)
- [Uso](#)
- [Estructura del Proyecto](#)
- [API Endpoints](#)
- [Seeds y Datos](#)
- [Pruebas](#)
- [Despliegue en Producción](#)
- [Contribuir](#)
- [Licencia](#)

Características

Sistema de Aprendizaje

- 12 Unidades de contenido** organizadas por niveles CEFR (A1-C2)
- Gramática completa** con explicaciones detalladas y ejemplos
- Vocabulario por categorías** con definiciones, ejemplos y sinónimos
- Ejercicios de escritura** con feedback automático inteligente
- Práctica de oraciones** con corrección en tiempo real

Mini Juegos Educativos

Juego	Descripción	Niveles
 Word Scramble	Ordena las letras para formar palabras	A1-C2
 Hangman	Adivina la palabra letra por letra	A1-C2
 Memory Match	Encuentra pares inglés-español	A1-C2
 Fill the Gaps	Completa oraciones con la palabra correcta	A1-C2
? Quick Quiz	Preguntas rápidas de gramática y vocabulario	A1-C2
 Reading Comprehension	Lecturas con preguntas de comprensión	A1-C1

Juego	Descripción	Niveles
 Speed Typing	Escribe frases lo más rápido posible	Fácil-Difícil

Sistema de Gamificación

- **Sistema de puntos** por completar actividades y juegos
- **Rachas diarias** (streaks) con bonificaciones
- **Insignias y logros** desbloqueables
- **Tabla de clasificación** semanal y mensual
- **Niveles de experiencia** con progresión

Seguimiento de Progreso

- **Dashboard personalizado** con estadísticas
- **Historial de actividades** y puntuaciones
- **Gráficos de progreso** por unidad y habilidad
- **Reporte de errores frecuentes** para reforzar áreas débiles

Sistema de Usuarios

- Registro e inicio de sesión seguro
- **Email de bienvenida** automático a nuevos usuarios
- Recuperación de contraseña por email
- Perfiles personalizables

Experiencia de Usuario

- **Modo oscuro/claro** con persistencia
- Diseño **100% responsive** (móvil, tablet, desktop)
- Interfaz intuitiva con Bootstrap 5
- Animaciones suaves y feedback visual

Tecnologías

Backend

- **Python 3.12** - Lenguaje principal
- **Flask 3.0** - Framework web
- **SQLAlchemy** - ORM para base de datos
- **Flask-Login** - Autenticación de usuarios
- **Flask-Mail** - Sistema de emails
- **Flask-Migrate** - Migraciones de base de datos
- **Gunicorn** - Servidor WSGI para producción

Frontend

- **HTML5 / CSS3 / JavaScript**
- **Bootstrap 5.3** - Framework CSS

- **Font Awesome 6** - Iconografía
- **Chart.js** - Gráficos de progreso

Base de Datos

- **PostgreSQL 16** - Base de datos principal
- Índices optimizados para consultas rápidas

Infraestructura

- **Nginx** - Proxy reverso y servidor estático
- **Systemd** - Gestión de servicios
- **Let's Encrypt** - Certificados SSL/TLS
- **Ubuntu Server** - Sistema operativo

Instalación

Prerrequisitos

- Python 3.10+
- PostgreSQL 14+
- pip y virtualenv
- Git

Clonar el repositorio

```
git clone https://github.com/Ache2503/english-review.git
cd english-review
```

Crear entorno virtual

```
python3 -m venv venv
source venv/bin/activate # Linux/Mac
# o en Windows: venv\Scripts\activate
```

Instalar dependencias

```
pip install -r requirements.txt
```

Crear base de datos PostgreSQL

```
sudo -u postgres psql
```

```
# En la consola de PostgreSQL:
CREATE DATABASE ingles_db;
CREATE USER ingles_user WITH ENCRYPTED PASSWORD 'tu_password_seguro';
GRANT ALL PRIVILEGES ON DATABASE ingles_db TO ingles_user;
\q
```

⚙ Configuración

Variables de entorno

Crear archivo `.env` en la raíz del proyecto:

```
# Flask
SECRET_KEY=tu_clave_secreta_muy_larga_y_segura
FLASK_ENV=production
FLASK_DEBUG=False

# Base de datos
DATABASE_URL=postgresql://ingles_user:tu_password@localhost:5432/ingles_db

# Email (Gmail)
MAIL_ENABLED=True
MAIL_SERVER=smtp.gmail.com
MAIL_PORT=587
MAIL_USE_TLS=True
MAIL_USERNAME=tu_email@gmail.com
MAIL_PASSWORD=tu_app_password
MAIL_DEFAULT_SENDER=tu_email@gmail.com
```

Nota: Para Gmail, necesitas crear una [contraseña de aplicación](#).

Ejecutar migraciones

```
export FLASK_APP=run.py
flask db upgrade
```

Cargar datos iniciales (seeds)

```
python seed_runner.py
```

Uso

Desarrollo local

```
source venv/bin/activate
python run.py
```

Abrir en: <http://localhost:5000>

Producción con Gunicorn

```
gunicorn --workers 4 --bind 0.0.0.0:8000 run:app
```

Estructura del Proyecto

```
english-review/
├── app/
│   ├── __init__.py          # Factory de la aplicación
│   ├── extensions.py        # Extensiones Flask (db, login, mail)
│   ├── models.py           # Modelos SQLAlchemy (50+ tablas)
│   └── routes/
│       ├── auth.py          # Login, registro, logout
│       ├── main.py          # Páginas principales
│       ├── dashboard.py     # Panel del usuario
│       ├── units.py         # Contenido de unidades
│       ├── practice.py      # Ejercicios de práctica
│       ├── quiz.py          # Sistema de quizzes
│       ├── games.py         # Mini juegos (7 juegos)
│       ├── badges.py        # Sistema de insignias
│       ├── flashcards.py    # Tarjetas de vocabulario
│       ├── reading.py       # Lecturas
│       ├── explanations.py  # Explicaciones detalladas
│       └── challenges.py    # Desafíos y puntos
│   ├── services/
│       ├── feedback.py      # Análisis de texto inteligente
│       ├── streaks.py       # Sistema de rachas
│       └── email_service.py # Servicio de emails
│   └── templates/
│       ├── base.html        # Template base con navbar
│       ├── index.html       # Página de inicio
│       ├── dashboard.html   # Panel del usuario
│       ├── games/           # Templates de juegos
│       ├── auth/            # Login y registro
│       └── ...
├── seeds/
│   ├── unit_data.json       # Datos de unidades
│   └── extended_unit_data.json
├── logs/                    # Logs de la aplicación
├── config.py                # Configuración
├── run.py                   # Punto de entrada
└── requirements.txt         # Dependencias Python
```

```
|— seed_*.py
|— README.md
```

```
# Scripts de seed
```

API Endpoints

Autenticación

Método	Endpoint	Descripción
GET/POST	/auth/login	Iniciar sesión
GET/POST	/auth/register	Registrar usuario
GET	/auth/logout	Cerrar sesión

Unidades y Contenido

Método	Endpoint	Descripción
GET	/units/	Lista de unidades
GET	/units/<id>	Detalle de unidad
GET	/units/<id>/grammar	Gramática de unidad
GET	/units/<id>/vocabulary	Vocabulario de unidad

Mini Juegos

Método	Endpoint	Descripción
GET	/games/	Lista de juegos
GET	/games/word-scramble	Word Scramble
POST	/games/word-scramble/submit	Guardar puntuación
GET	/games/hangman	Hangman
POST	/games/hangman/submit	Guardar puntuación
GET	/games/memory	Memory Match
GET	/games/fill-gaps	Fill the Gaps
GET	/games/quick-quiz	Quick Quiz
GET	/games/reading	Reading Comprehension
GET	/games/speed-typing	Speed Typing

Práctica

Método	Endpoint	Descripción
--------	----------	-------------

Método	Endpoint	Descripción
POST	/practice/api/analyze	Analizar texto
POST	/practice/submit	Enviar ejercicio

Seeds y Datos

Scripts de seed disponibles

```
# Ejecutar todos los seeds
python seed_runner.py

# O ejecutar individualmente:
python seed_db.py           # Unidades base
python seed_db_extended.py  # Contenido extendido
python seed_badges.py       # Insignias
python seed_flashcards.py   # Tarjetas de vocabulario
python seed_explanations.py # Explicaciones
python seed_games_content.py # Contenido de juegos
python seed_motivational_messages.py # Mensajes motivacionales
python seed_verb_tenses.py  # Tiempos verbales
python seed_topic_explanations.py # Explicaciones de temas
```

Contenido incluido

Tipo	Cantidad
Unidades	12
Temas de gramática	50+
Vocabulario	500+ palabras
Flashcards	200+
Preguntas de quiz	45+
Lecturas	9
Frases para typing	43
Insignias	25+
Tiempos verbales	480 conjugaciones

Pruebas

Ejecutar todas las pruebas

```
python run_all_tests.py
```

Pruebas individuales

```
# Sistema de feedback
python test_feedback_system.py

# Integración de rutas
python test_integration.py

# Verificar responsividad
python check_responsiveness.py

# Verificar modo oscuro
python verify_dark_mode.py
```

Cobertura actual

-  Sistema de feedback: 18/18 pruebas
-  Integración: 3/3 pruebas
-  Tasa de éxito: 100%

Despliegue en Producción

Configuración con Nginx + Gunicorn

1. Crear servicio systemd

```
sudo nano /etc/systemd/system/ingles.service
```

```
[Unit]
Description=Gunicorn service for English Review Platform
After=network.target

[Service]
User=www-data
Group=www-data
WorkingDirectory=/var/www/english-review
Environment="PATH=/var/www/english-review/venv/bin"
ExecStart=/var/www/english-review/venv/bin/gunicorn \
    --workers 4 \
    --worker-class sync \
    --bind unix:/var/www/english-review/ingles.sock \
    --timeout 30 \
```



```
--access-logfile /var/log/english-review/access.log \  
--error-logfile /var/log/english-review/error.log \  
run:app
```

[Install]

WantedBy=multi-user.target

2. Configurar Nginx

```
server {  
    listen 80;  
    server_name ingles.jaripeo.online;  
    return 301 https://$server_name$request_uri;  
}  
  
server {  
    listen 443 ssl;  
    server_name ingles.jaripeo.online;  
  
    ssl_certificate  
/etc/letsencrypt/live/ingles.jaripeo.online/fullchain.pem;  
    ssl_certificate_key  
/etc/letsencrypt/live/ingles.jaripeo.online/privkey.pem;  
  
    location / {  
        proxy_pass http://unix:/var/www/english-review/ingles.sock;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
        proxy_set_header X-Forwarded-Proto $scheme;  
    }  
  
    location /static {  
        alias /var/www/english-review/app/static;  
        expires 30d;  
    }  
}
```

3. Habilitar y arrancar servicios

```
sudo systemctl daemon-reload  
sudo systemctl enable ingles.service  
sudo systemctl start ingles.service  
sudo systemctl restart nginx
```

4. Verificar estado

```
sudo systemctl status ingles.service
curl -I https://ingles.jaripeo.online
```

Contribuir

Las contribuciones son bienvenidas. Por favor:

1. Fork el repositorio
2. Crea una rama para tu feature (`git checkout -b feature/AmazingFeature`)
3. Commit tus cambios (`git commit -m 'Add: nueva característica'`)
4. Push a la rama (`git push origin feature/AmazingFeature`)
5. Abre un Pull Request

Guías de estilo

- **Python:** PEP 8
- **Commits:** Conventional Commits
- **Documentación:** Docstrings en español

Licencia

Este proyecto está bajo la Licencia MIT. Ver [LICENSE](#) para más detalles.

Autor

Axel Hernández - [@Ache2503](#)

☆ Si este proyecto te fue útil, considera darle una estrella en GitHub ☆